

R2.06 Exploitation BDD

M. T. Pham, H. Evenas, E. Lemonnier, P. Adorni

minh-tan.pham@univ-ubs.fr

Déroulement

Rappel - Principales instructions SQL

- Langage de Définition des Données (LDD) : CREATE, DROP, ALTER
- Langage de Manipulation des Données (LMD)
 - Insertion, Mis à jour, Suppression: INSERT, UPDATE, DELETE
 - Interrogations: SELECT
 - Projection, Restriction, Tri (ORDER BY), ROWNUM
 - Union, Intersection, Différence ensembliste (UNION, INTERSECT, MINUS)
 - Produit, Jointures internes (JOIN)
 - Jointures externes (LEFT/RIGHT/FULL JOIN)
 - Sous-requêtes (IN, EXISTS)
 - Fonctions de groupe (COUNT, SUM) et Regroupement (GROUP BY, HAVING)
 - Division
 - Vues (VIEW)
- Langage de Contrôle des Données (LCD)
 - Contrôle d'accès aux données GRANT, REVOKE

*R1.05-P2
(Oracle)*

*R2.06
(MySQL, Oracle)*

Déroulement de R2.06

- Période 3 : 1TDi+ 1TP (avec PC perso) par semaine * 6 semaines (Requêtes et Vues)
 - Sous-requêtes et Compléments des jointures
 - Fonctions de groupe et Regroupement
 - Division
 - Vues
- Période 4 : 1TP (avec PC perso) par semaine * 6 semaines (Administration BD)
 - Administration BD
- **Evaluation:**
 - Présence obligatoire en Cours, TDs et TP
 - Rendus des TP hebdo (à déposer sur Moodle)
 - Contrôle final en P3 : écrit (Calculatrice + A4 recto-verso autorisé)
 - Contrôle continu en P4 : modalités à préciser

R2.06 Exploitation BDD

M. T. Pham, H. Evenas, E. Lemonnier, P. Adorni

minh-tan.pham@univ-ubs.fr

Cours 1

Sous-requêtes et Rappels

Plan

Sous-requêtes (mapping imbriqué)

Test de valeur

Test d'existence

Sous-requêtes vs Jointures

Compléments sur les jointures

Rappel : Tri + Limitation (déjà vu en R1.05)

Produit (nomPro(1), prix, ...)

Quels sont les dix produits les plus chers ?



```
SELECT nomPro, prix
FROM ( SELECT *
      FROM      Produit
      ORDER BY  prix DESC
      )
WHERE ROWNUM <= 10 ;
```

Une **sous-requête** (ou un mapping imbriqué) → une requête qui contient une (des) autre(s) requête(s)

Remarque

ATTENTION: Oracle SQL vs MySQL

```
SELECT DISTINCT nomPro
FROM ( SELECT *
      FROM      Produit
      ORDER BY  prix DESC
    )
WHERE nomPro LIKE 'S%';
```

```
SELECT DISTINCT nomPro
FROM ( SELECT *
      FROM      Produit
      ORDER BY  prix DESC
    ) AS subQuery
WHERE nomPro LIKE 'S%';
```

MySQL: Alias obligatoire pour la sous-requête suivant le FROM

```
SELECT nomPro
FROM Produit
WHERE nomPro LIKE 'S%'
AND ROWNUM <= 10;
```

```
SELECT nomPro
FROM Produit
WHERE nomPro LIKE 'S%'
LIMIT 10;
```

MySQL: LIMIT au lieu de WHERE ROWNUM

Test avec une valeur retournée



Employe (nom(1), prenom, fonction, salaire,)

Quels sont les employés qui font le même travail qu'Obélix ?

```
SELECT DISTINCT nom
FROM Employe
WHERE fonction = ( SELECT fonction
                   FROM   Employe
                   WHERE  UPPER(nom) = 'OBELIX'
                 ) ;
```

Remarque : Dans ce cas, on suppose qu'il y a un seul 'OBELIX' dans notre base.

Test avec plusieurs valeurs (appartenance à une liste)

Employe (nom(1), fonction, salaire, lEntreprise = @Entreprise[noEnt]),)

Entreprise (noEnt(1), chiffreAffaire,

Quels sont les employés qui travaillent dans une entreprise dont le chiffre d'affaire dépasse 1 million ?

```
SELECT DISTINCT nom
FROM   Employe
WHERE  lEntreprise IN ( SELECT noEnt
                        FROM   Entreprise
                        WHERE  chiffreAffaire > 1000000
                        ) ;
```

Test avec plusieurs valeurs (non appartenance à une liste)

Employe (nom(1), fonction, salaire, lEntreprise = @Entreprise[noEnt]),)

Entreprise (noEnt(1), chiffreAffaire,

Quels sont les Employés qui ne sont affectés à aucune entreprise ?

```
SELECT DISTINCT nom
FROM   Employe
WHERE  lEntreprise NOT IN ( SELECT noEnt
                             FROM   Entreprise
                             ) ;
```

Test d'existence (au moins un tuple retourné)

Employe (nom(1), fonction, salaire, lEntreprise = @Entreprise[noEnt]),)

Entreprise (noEnt(1), chiffreAffaire,

Quels sont les fonctions relatives à au moins 2 employés ?

```
SELECT DISTINCT fonction
FROM   Employe E
WHERE  EXISTS ( SELECT *
                  FROM   Employe
                  WHERE  fonction = E.fonction
                  AND    nom != E.nom
                ) ;
```

Test de contenu vide (aucun tuple retourné)

Employe (nom(1), fonction, salaire, lEntreprise = @Entreprise[noEnt]),)

Entreprise (noEnt(1), chiffreAffaire,

Quels sont les entreprises ou tous les employés gagnent moins de 1800 euros ?

```
SELECT DISTINCT noEnt
FROM   Entreprise
WHERE  NOT EXISTS ( SELECT *
                     FROM   Employe
                     WHERE   lEntreprise = noEnt
                     AND     salaire >= 1800
                     ) ;
```

Plan

Sous-requêtes (mapping imbriqué)

Test de valeur

Test d'existence

Sous-requêtes vs Jointures

Compléments sur les jointures

Sous-requêtes vs Jointures

On considère le schéma relationnel suivant :

Ouvrage (idOuvrage (1), titre, unAuteur = @Auteur.idAuteur (NN), anneeAchat)

Auteur (idAuteur (1), nom (NN), prenom, nationalite, anneeNaissance)

Client (idClient (1), nomClient (NN), adresse)

Emprunt (unClient = @Client.idClient (1), unOuvrage = Ouvrage.idOuvrage (1), dateEmprunt)

Sous-requêtes vs Jointures

Quels sont les titres des ouvrages écrits par un anglais ?

Avec une sous-interrogation

```
SELECT DISTINCT titre
FROM   Ouvrage
WHERE  unAuteur IN ( SELECT idAuteur
                      FROM   Auteur
                      WHERE    UPPER(nationalite) = 'ANGLAIS'
                      );
```

Avec une jointure

```
SELECT DISTINCT titre
FROM   Ouvrage, Auteur
WHERE  unAuteur = idAuteur
AND    UPPER(nationalite) = 'ANGLAIS';
```

Sous-requêtes vs Jointures

Quels sont les noms des clients ayant emprunté l'ouvrage dont le titre est Le Petit Prince ?

Avec une sous-interrogation

```
SELECT DISTINCT nomClient
FROM Client
WHERE idClient IN (SELECT unClient
                    FROM Emprunt
                    WHERE unOuvrage = ( SELECT idOuvrage
                                         FROM Ouvrage
                                         WHERE UPPER(titre) = 'LE PETIT PRINCE'
                                       )
                  );
```

Avec une jointure de 3 tables

```
SELECT DISTINCT nomClient
FROM Client, Emprunt, Ouvrage
WHERE idClient = unClient
AND unOuvrage = idOuvrage
AND UPPER(titre) = 'LE PETIT PRINCE';
```


Sous-requêtes vs Jointures

Quels sont les identifiants des clients qui n'ont emprunté aucun ouvrage

Avec une sous-interrogation

```
SELECT idClient
FROM   Client
WHERE  idClient NOT IN ( SELECT DISTINCT unClient
                        FROM   Emprunt
                        ) ;
```

Avec une différence

```
SELECT idClient
FROM   Client
MINUS
SELECT  unClient
FROM    Emprunt;
```

Sous-requêtes vs Jointures

Quels sont les noms portés à la fois par un auteur et par un client ?

Avec une sous-interrogation (EXISTS)

```
SELECT DISTINCT UPPER(nom)
FROM   Auteur
WHERE  EXISTS ( SELECT *
                  FROM   Client
                  WHERE  UPPER(nomClient) = UPPER(nom) );
```

Ou avec IN

```
SELECT DISTINCT UPPER(nom)
FROM   Auteur
WHERE  UPPER(nom) IN (SELECT UPPER(nomClient)
                       FROM   Client );
```

Ou avec une jointure aussi !!!

Sous-requêtes vs Jointures

Quels sont les auteurs qui ont écrit au moins 2 ouvrages ?

Avec EXISTS

```
SELECT DISTINCT O1.unAuteur
FROM   Ouvrage O1
WHERE  EXISTS ( SELECT *
                  FROM   Ouvrage O2
                  WHERE  O2.unAuteur = O1.unAuteur
                  AND    O2.idOuvrage != O1.idOuvrage
                ) ;
```

Ou avec IN

```
SELECT DISTINCT O1.unAuteur
FROM   Ouvrage O1
WHERE  O1.unAuteur IN (SELECT O2.unAuteur
                        FROM   Ouvrage O2
                        WHERE  O2.idOuvrage != O1.idOuvrage
                        ) ;
```

Sous-requêtes vs Jointures

Quels sont les auteurs qui ont écrit au moins 2 ouvrages ?

Avec EXISTS

```
SELECT DISTINCT O1.unAuteur
FROM   Ouvrage O1
WHERE  EXISTS ( SELECT *
                  FROM   Ouvrage O2
                  WHERE  O2.unAuteur = O1.unAuteur
                  AND    O2.idOuvrage != O1.idOuvrage
                );
```

Ou avec une auto-jointure (très efficace)

```
SELECT DISTINCT O1.unAuteur
FROM   Ouvrage O1, Ouvrage O2
WHERE  O1.unAuteur = O2.unAuteur
AND    O2.idOuvrage != O1.idOuvrage;
```

Plan

Sous-requêtes (mapping imbriqué)

Test de valeur

Test d'existence

Sous-requêtes vs Jointures

Compléments sur les jointures

Compléments sur les jointures

Compagnie (idComp (1), nomComp, pays, estLowCost)

Pilote (idPilote(1), nomPilote, nbHVol, compPil=@Compagnie(idComp))

TypeAvion (idTypeAvion(1),nbPassagers)

Qualification (unPilote=@Pilote(idPilote)(1),unTypeAvion=@TypeAvion(idTypeAvion)(1))

Avion (idAvion(1), leTypeAvion=@TypeAvion(idTypeAvion)(NN), compAv=@Compagnie(idComp)(NN))

Compagnie

idComp	nomComp	pays	estLowCost
1	Air France	France	0
2	Corsair International	France	0
3	EasyJet	Angleterre	1
4	American Airlines	Etats-Unis	0
5	Ryanair	Irlande	1

TypeAvion

idTypeAvion	nbPassagers
A320	174
A350	324
B747	279

Qualification

unPilote	unTypeAvion
1	A320
1	A350
2	A320
2	B747
3	A320
4	A320
4	A350
4	B747
5	A350
5	A320
7	A350
7	B747

Pilote

idPilote	nomPilote	nbHVol	compPil
1	Ridard	1500	1
2	Naert	450	3
3	Godin	450	5
4	Fleurquin	3000	1
5	Pham	900	4
6	Kerbellec	900	
7	Kamp	3000	4

Avion

idAvion	leTypeAvion	compAv
1	A320	1
2	A320	3
3	A350	1
4	A320	2
5	B747	1
6	A350	4
7	B747	4
8	A320	5
9	A320	5

Compléments sur les jointures

Pour chaque pilote désigné par son nom, afficher le nom de sa compagnie.



Jointure relationnelle

```
SELECT nomPilote , nomComp  
FROM Pilote , Compagnie  
WHERE compPil = idComp  
;
```



Jointure SQL2

```
SELECT nomPilote , nomComp  
FROM Pilote  
JOIN Compagnie ON compPil = idComp  
;
```

RESULT

NOMPILOTE	NOMCOMP
Ridard	Air France
Naert	EasyJet
Godin	Ryanair
Fleurquin	Air France
Pham	American Airlines
Kamp	American Airlines

Attention : les résultats sont donnés avec Oracle SQL, il vous sera demandé de vérifier avec MySQL en TP

Compléments sur les jointures

Pour chaque pilote désigné par son nom, afficher le nom de sa compagnie et toutes ses qualifications.



Double jointure relationnelle

```
SELECT nomPilote, nomComp, unTypeAvion
FROM Pilote, Compagnie, Qualification
WHERE compPil = idComp
AND idPilote = unPilote
;
```



Double jointure SQL2

```
SELECT nomPilote, nomComp, unTypeAvion
FROM Pilote
JOIN Compagnie ON compPil = idComp
JOIN Qualification ON idPilote = unPilote
```

RESULT

NOMPILOTE	NOMCOMP	UNTYPEAVION
Ridard	Air France	A320
Ridard	Air France	A350
Naert	EasyJet	A320
Naert	EasyJet	B747
Godin	Ryanair	A320
Fleurquin	Air France	A320
Fleurquin	Air France	A350
Fleurquin	Air France	B747
Pham	American Airlines	A350
Pham	American Airlines	A320
Kamp	American Airlines	A350
Kamp	American Airlines	B747

Compléments sur les jointures

Pour chaque pilote (éventuellement sans compagnie) désigné par son nom, afficher le nom de sa compagnie (si elle existe).



Jointure externe relationnelle

```
SELECT nomPilote , nomComp
FROM Pilote , Compagnie
WHERE compPil = idComp(+)
;
```



Jointure externe SQL2

```
SELECT nomPilote , nomComp
FROM Pilote
LEFT JOIN Compagnie ON compPil = idComp
;
```

RESULT

NOMPILOTE	NOMCOMP
Fleurquin	Air France
Ridard	Air France
Naert	EasyJet
Kamp	American Airlines
Pham	American Airlines
Godin	Ryanair
Kerbellec	

Compléments sur les jointures

Pour chaque compagnie (éventuellement sans pilote) désignée par son nom, afficher le nom de ses pilotes (s'il en existe).



Jointure externe relationnelle

```
SELECT nomPilote , nomComp
FROM Pilote , Compagnie
WHERE compPil(+) = idComp
ORDER BY nomComp
;
```



Jointure externe SQL2

```
SELECT nomPilote , nomComp
FROM Pilote
      RIGHT JOIN Compagnie ON compPil = idComp
ORDER BY nomComp
;
```

RESULT

NOMPILOTE	NOMCOMP
Ridard	Air France
Fleurquin	Air France
Pham	American Airlines
Kamp	American Airlines
	Corsair International
Naert	EasyJet
Godin	Ryanair

Références

- <https://fr.wikipedia.org/>
- Cours BDD Anthony Ridard, <https://math-ridard.fr/>
- Christian Soutou, *Modélisation des bases de données : UML et les modèles entité-association*, 3^{ème} édition, Groupe Eyrolles.
- Christian Soutou, *SQL pour Oracle : Optimisation des requêtes et schémas*, 5^{ème} édition, Groupe Eyrolles.
- Elisabetta De Mria, Cours L2 Informatique, UFR Sciences, Université Côte d'Azur, <https://www.i3s.unice.fr/%18edemaria/>
- Oracle SQL Tutorials, <https://www.w3schools.com/sql/default.asp>
- MySQL Tutorials, <https://www.w3schools.com/MySQL/default.asp>